Creation serveur vpn raspberry pi 1 sur Alpine Linux
Par Domimoz

Je considère que Alpine linux est installé sur le raspberry pi avec un clé de capacité suffisante par exemple 2 Go
Il sera possible par la suite de copier l'installation sur une clé de 256 Mo puisque nous avons besoin de moins 150 Mo

- Nota Alpine linux peut tre installé facilement avec PI Imager une fois démarré il suffit de faire setup-alpine et il s'installera sur le disque

Installation open Vpn et easy-rsa

```
sudo apk add openvpn easy-rsa

→ utilisation de easy-rsa pour créer les certificats :

CA
├─ certificat de l'autorité
│
├─ certificat serveur
│  └─ clé serveur
│
└─ certificat client
    └── clé client

Emplacement des certificats dans /etc/easy-rsa
```

- **Initialiser la PKI** (Public Key Infrastructure)

```
sudo mkdir -p /etc/easy-rsa
sudo cp -r /usr/share/easy-rsa/* /etc/easy-rsa/
cd /etc/easy-rsa

sudo ./easyrsa init-pki
```

```
→ on obtient a ce stade une infrastructure du type :
pki/
├── private/
├── issued/
├── reqs/
├── certs_by_serial/
└── …
```

- **Creer la CA**

```
sudo ./easyrsa build-ca

→ la clé va s’inscrire :
pki/
├── ca.crt
└── private/
    └── ca.key
```

- **Certificat du serveur**

On va maintenant créer une demande de certificat pour le Raspberry :

```
sudo ./easyrsa gen-req server nopass

pki/
├── private/
│   └── server.key       ← clé privée
│
└── reqs/
    └── server.req       ← demande de certificat
```

- **signer le certificat du serveur**

Maintenant, on demande à notre CA de signer le certificat **server**.

Lance :

```
sudo ./easyrsa sign-req server server
```

- **Creation clé Diffie-Hellman**

Pour que notre serveur OpenVPN, génère maintenant les paramètres DH

```
sudo ./easyrsa gen-dh
```

```
pki/
├── dh.pem  ← Creation de la clé
```

- **Générer la clé tls-crypt (dans /etc/openvpn/ta.key)**

Depuis /etc/easy-rsa :

```
sudo openvpn --genkey tls-crypt /etc/openvpn/ta.key
```

- **Generer le certificat Client :**

```
sudo ./easyrsa gen-req client1 nopass
```

```
→ creation des clé dans :
* req: /etc/easy-rsa/pki/reqs/client1.req
* key: /etc/easy-rsa/pki/private/client1.key

→ le faire signer :
```

```
sudo ./easyrsa sign-req client client1
```

- **Debut de la configuration du reseau OPENVPN**

```
definition du reseau VPN 10.8.0.0/24
```

- **Creation d’un repertoire de travail**

```
mkdir -p /etc/openvpn/server
```

- **Recuperation des certificats**

```
sudo cp /etc/easy-rsa/pki/ca.crt /etc/openvpn/server/
sudo cp /etc/easy-rsa/pki/issued/server.crt /etc/openvpn/server/
sudo cp /etc/easy-rsa/pki/private/server.key /etc/openvpn/server/
sudo cp /etc/easy-rsa/pki/dh.pem /etc/openvpn/server/
sudo cp /etc/openvpn/ta.key /etc/openvpn/server/
```

Verification :

```
sudo ls -l /etc/openvpn/server/
```

Il doit y avoir :

```
ca.crt
dh.pem
server.crt
server.key
ta.key
```

- **Configuration OPENVPN**

On va utiliser :

- VPN : **10.8.0.0/24**
- IP du serveur VPN : **10.8.0.1**
- clients : **10.8.0.2**, **10.8.0.3**, etc.
- protocole : UDP
- port : **1194**
- **tls-crypt**
- chiffrement moderne
- accès au réseau maison **192.168.1.0/24**

→ Creation du fichier

```
sudo vi /etc/openvpn/server/server.conf
```

Contenu :

```
port 1194
proto udp
dev tun

server 10.8.0.0 255.255.255.0

ca /etc/openvpn/server/ca.crt
cert /etc/openvpn/server/server.crt
key /etc/openvpn/server/server.key
dh /etc/openvpn/server/dh.pem

tls-crypt /etc/openvpn/server/ta.key

topology subnet

push "route 192.168.1.0 255.255.255.0"

keepalive 10 120

user nobody
group nogroup

persist-key
persist-tun

verb 3

data-ciphers AES-256-GCM:AES-128-GCM
```

- **Activer tun :**

```
sudo modprobe tun
```

- **Lancer le serveur vpn**

```
sudo openvpn --config /etc/openvpn/server/server.conf
```

→ serveur lancé (voir Initialization Sequence Completed)

→ vérification en ouvrant une 2e fenetre ssh :

```
ip addr show tun0
```

Verifier quelque chose comme :

```
tun0:
    inet 10.8.0.1/24
```

➢ **Creation du fichier client client1.ovpn**

→ On laisse la fenetre où openvpn est lancé, et on agit à partir d'une autre fenetre ssh

Principe le fichier contiendra :

- le certificat CA ;
- le certificat **client1** ;
- la clé privée **client1** ;
- la clé **tls-crypt** ;
- l'adresse publique de la box

- **Verification des Fichiers certificats**

Verifier les 4 fichiers :

```
tree /etc/easy-rsa/pki
ls /etc/openvpn/

/etc/easy-rsa/pki/ca.crt
/etc/easy-rsa/pki/issued/client1.crt
/etc/easy-rsa/pki/private/client1.key
/etc/openvpn/ta.key
```

1.
   - **Creation du fichier**

```
sudo vi /etc/openvpn/client1.ovpn
```

   -
   - **Nota corriger ci dessous** remote 1.2.3.4 1120 : remplacer par @ip de la box / port choisi pour le NAT
   - **Contenu du fichier à mettre :**

```
client
dev tun
proto udp

remote 1.2.3.4 1120

resolv-retry infinite
nobind

persist-key
persist-tun

remote-cert-tls server

auth-nocache

data-ciphers AES-256-GCM:AES-128-GCM

verb 3

<ca>
```

```
</ca>

<cert>
</cert>

<key>
</key>

<tls-crypt>
</tls-crypt>
```

- **Creation du Script pour remplir les champs**

```
vi /etc/openvpn/script.sh
```

Contenu

```
sudo sh -c '{
cat <<EOF
client
dev tun
proto udp
remote 1.2.3.4 1120
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
auth-nocache
data-ciphers AES-256-GCM:AES-128-GCM
verb 3

<ca>
EOF
cat /etc/easy-rsa/pki/ca.crt
```

```
cat <<EOF
</ca>

<cert>
EOF
cat /etc/easy-rsa/pki/issued/client1.crt
cat <<EOF
</cert>

<key>
EOF
cat /etc/easy-rsa/pki/private/client1.key
cat <<EOF
</key>

<tls-crypt>
EOF
cat /etc/openvpn/ta.key
cat <<EOF
</tls-crypt>
EOF
} > /etc/openvpn/client1.ovpn'
```

Mettre le fichier en executable :

```
chmod +x /etc/openvpn/script.sh
```

- **Gestion IP_FORWARD**

```
Pour que ip_forward=1 au demarrage :
vi /etc/sysctl.conf
Ajouter la ligne :
net.ipv4.ip_forward=1
```

Pour que ip_forward=1 maintenant

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Verification

```
cat /proc/sys/net/ipv4/ip_forward
```

Il faut obtenir :1

Gestion du parefeu

- **Installer iptables**

```
apk add iptables
```

- **Creation du fichier iptables**

```
vi /etc/iptables/rules.sh
```

Contenu du fichier

```
#!/bin/sh

# Nettoyage des règles précédentes
iptables -F
iptables -t nat -F

# NAT du réseau VPN vers eth0
iptables -t nat -A POSTROUTING \
    -s 10.8.0.0/24 \
    -o eth0 \
    -j MASQUERADE

# Autoriser le trafic VPN -> réseau local
iptables -A FORWARD \
    -i tun0 \
    -o eth0 \
    -s 10.8.0.0/24 \
```

```
    -d 192.168.1.0/24 \
    -j ACCEPT

# Autoriser les réponses réseau local -> VPN
iptables -A FORWARD \
    -i eth0 \
    -o tun0 \
    -m conntrack \
    --ctstate ESTABLISHED,RELATED \
    -j ACCEPT
```

- passer le fichier executable :

```
chmod +x /etc/iptables/rules.sh
```

pour demarrage faire:

```
vi /etc/init.d/iptables-vpn

contenu :
#!/sbin/openrc-run

name="iptables-vpn"
description="iptables rules for OpenVPN"

depend() {
    need net
    after firewall
}

start() {
    ebegin "Loading OpenVPN iptables rules"
    /etc/iptables/rules.sh
    eend $?
}
```

passer en executable :

```
chmod +x /etc/init.d/iptables-vpn
```

le mettre en demarrage automatique :

```
rc-update add iptables-vpn default
```

- **Creer un lien symbolique pour eviter les pb de configuration (au lancement openvpn il doit retrouver son fichier conf)**

```
sudo ln -s /etc/openvpn/server/server.conf
/etc/openvpn/openvpn.conf
```

- **Ajouter openvpn au démarrage automatique**

```
sudo rc-update add openvpn default

Verification
rc-update show | grep -E 'openvpn|iptables'
         iptables-vpn |      default
              openvpn |      default
```

SECURITE

```
Installation de fail2ban

apk add fail2ban

Demarrage du fail2ban
rc-service fail2ban start
```

demarrage automatique:
rc-update add fail2ban

- **Configurer une regle (Jail)**

cat /etc/fail2ban/jail.d/alpine-ssh.conf
contenu :

```
[sshd]
enabled  = true
filter   = alpine-sshd
port     = ssh
logpath  = /var/log/messages
maxretry = 10

[sshd-ddos]
enabled  = true
filter   = alpine-sshd-ddos
port     = ssh
logpath  = /var/log/messages
maxretry = 10
```

→ à ce stade on ne surveille que le port 22 il faut maintenant surveiller le port d'entré VPN ici 5412 (à ajuster) et on en profite pour créer une action mqtt qu'on publiera avec mosquitto_pub

- **creation du fichier de surveillance openvpn**
- **et du message mqtt**

Creer le fichier jail.local :
vi /etc/fail2ban/jail.local
contenu :

```
[openvpn]
enabled  = true
port     = 5412
protocol = udp
filter   = openvpn
logpath  = /var/log/messages
maxretry = 3
findtime = 600
bantime  = 3600

[sshd]
enabled = true
port    = ssh
logpath = /var/log/messages
backend = %(sshd_backend)s
# Ajout de l'action MQTT :
action  = iptables-multiport[name=SSH, port="ssh",
protocol=tcp]
          mqtt-notify[name=ssh]
```

- **Commande pour voir les jails et leurs status:**

```
fail2ban-client status
→ Status
|- Number of jail:   3
`- Jail list:    openvpn, sshd, sshd-ddos

fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed:  0
|  |- Total failed:  2
|  `- File list: /var/log/messages
`- Actions
   |- Currently banned:  0
   |- Total banned:  0
   `- Banned IP list:
```

- **Transmission de la notification avec mosquitto à home assistant**

```
apk add mosquitto-clients

vi /etc/fail2banaction.d/mqtt-notify.conf

contenu :
actionban = mosquitto_pub -h @IP-HomeAssitant -u user-mosquitto -P Motdepasse \
              -t "alpine/fail2ban/<name>" \
              -m '{"ip": "<ip>", "jail": "<name>", "failures": "<failures>", "status": "banned"}'

# Optionnel : notifier lors du débannissement
actionunban = mosquitto_pub -h IP_BROKER -u USER -P PASS \
                -t "alpine/fail2ban/<name>" \
                -m '{"ip": "<ip>", "jail": "<name>", "status": "unbanned"}'
```

- **Parametrage Home assistant / ajout entité**

```
Dans /homeassistant/configuration.yaml

ajouter :
mqtt:
  - sensor:
      name: "AlpineOpenVpnAlert"
      state_topic: "alpine/fail2ban/+"
      value_template: "{{ value_json.ip }}"
      json_attributes_topic: "alpine/fail2ban/+"
      icon: mdi:shield-alert
```

→ rebooter home assistant

L’installation est maintenant terminée
en faisant un df -h on constatera des utilisation comme suis :

```
df -h
Filesystem                Size      Used Available Use% Mounted on
devtmpfs                 10.0M         0     10.0M   0% /dev
shm                     213.6M         0    213.6M   0% /dev/shm
/dev/mmcblk0p2          165.5M    116.7M     34.6M  77% /
tmpfs                    85.5M    132.0K     85.3M   0% /run
/dev/mmcblk0p1           36.8M     22.7M     14.1M  62% /boot
tmpfs                   213.6M         0    213.6M   0% /tmp
tmpfs                    50.0M    100.0K     49.9M   0% /var/log
165.5M    116.7M     34.6M  77% /
```

soit une utilisation de 116,7+22,7 = **139,4 Mo**

Vous pouvez par copier/coller utiliser gparted et copier les 2 partitions mmcblk0P[01;02] sur une clé 256 Mo pour

Ajustement FSTAB pour économiser la carte SD

Pour éviter des lectures écritures importantes
il est possible d’ajuster le fstab pour le repertoire /var/log soit ecrit sur la ram

- sudo vi /etc/fstab

Ajouter la ligne :

```
tmpfs /var/log tmpfs defaults,noatime,nosuid,nodev,size=50M 0 0
```

On peut directement activer sans reboot par
mount /var/log
nota : - A chaque reboot ce sera pris en compte puisque le fichier fstab sera lu

Parcequ’on ecrit plus sur le disque mais dans la ram il faut utiliser un outil de gestion logrotate pour que le volume d’écriture n’exède pas et ne provoque pas d’erreur

- installation de logrotate

apk add logrotate

- configuration de /etc/logrotate.conf

Ajout d’une ligne de sécurité lorsqu’un fichier atteint 10 Mo :

```
# Safety limit: rotate immediately if a log reaches 10M
size 10M
```

- le fichier logrotate.conf sera alors :

```
# see "man logrotate" for details
# rotate log files weekly
weekly
# Safety limit: rotate immediately if a log reaches 10M
size 10M
# keep 4 weeks worth of backlogs
rotate 4
# create new (empty) log files after rotating old ones
create
# use date as a suffix of the rotated file
dateext
# exclude alpine files
tabooext + .apk-new
# uncomment this if you want your log files compressed
compress
# apk packages drop log rotation information into this directory
include /etc/logrotate.d
# system-specific logs may be also be configured here.
```

- Conclusion La configuration globale est suffisante on peut affiner pour chaque dossier dans /etc/logrotate.d/

Mettre le systeme sur une clé 256 Mo

En faisant
df -h
on obtient

```
/dev/mmcblk0p2          165.5M    116.7M     34.6M  77% /
/dev/mmcblk0p1           36.8M     22.7M     14.1M  62% /boot
```

Ici on se rend compte que tout l’emsemble prend 139,4 Mo
il suffit d’utiliser gparted de mettre la memoire qui a le systeme et l’autre carte de 256 Mo et de faire un copier coller des partitions
Nota : alpine linux a besoin de plus de 256 Mo pour s’installer mais on peut le faire par copier coller tel que décrit ci dessus

Home assistant est capable de vous transmettre l’adresse ip

- **Gerer l’adresse IP exterieure**

créer l’entité dans : /homeassistant/configuration.yaml

Ajouter le contenu :

```
sensor:
  - platform: rest
    name: "External_IP"
    resource: "https://api.ipify.org/?format=json"
    value_template: "{{ value_json.ip }}"
```

Au reboot l’entité « External_IP » sera créée on pourra la gerer dans une automation pour la recevoir par notification